COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

23IT721 - FULL STACK DEVELOPMENT LABORATORY

PROJECT

DIGITAL PAYMENT MANAGEMENT SYSTEM (DigitalPay)



OCTOBER 2026

Submitted by

2303717620521055	THIRAJ V S
2303717620522061	YUVASREE I
2303717620522002	ABITHA BEGAM M

COIMBATOREINSTITUTEOFTECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

BONAFIDE CERTIFICATE

Certified that this project report titled **"DIGITAL PAYMENT MANAGEMENT SYSTEM (DigitalPay)"** is the bonafide work of **2303717620521055 - THIRAJ V S, 2303717620522061 - YUVASREE I and 2303717620522002 - ABITHA BEGAM M** completed during the **academic year 2026-2027-Semester VII** for the project presentation of **23IT721 - FULL STACK DEVELOPMENT LABORATORY** under our supervision.

Certified that the candidates were examined by using the project work examination.

Review Members:

1. **Dr.C.Murale**
2. **Mr.Y.M.Ismankhan**

Coimbatore Institute of Technology-Coimbatore-14
Department of Information Technology

23IT721- FULL STACK DEVELOPMENT LABORATORY
CAT II - CASE STUDY EVALUATION FORM

Course: Semester: Date:

Roll No :
Name :
Title :

No.	Description	Marks	Marks Obtained
1	Problem Statement and Objective	20	
2	Event Handling	20	
3	Simulation/Results(AJAX/JQuery Operations)	30	
4	Presentation	10	
5	Report	10	
6	Viva	10	
	Total	100	

Observation / Suggestions

Name of the Staff

1. Dr.C.Murale
2. Mr.Y.M.Ismankhan

Signature of the Staff

TABLE OF CONTENTS

SECTION	TITLE	PAGE
—	Cover Page	1
—	Bonafide Certificate	2
—	CAT II Case Study Evaluation Form	3
—	Table of Contents	4–5
1	Introduction	6
2	Abstract, Objectives and Scope	6
3	Technology Stack	7
4	System Architecture	7
5	Home Page and Product Identity	8
6	Interactive Dashboard	8
7	Advanced Registration	9
8	Wallet and Payment Queue	9
9	Event-driven Payment Center	10
10	Transaction Center	11
11	Reports and Insights	11

TABLE OF CONTENTS — CONTINUED

SECTION	TITLE	PAGE
12	PHP Backend and User Management	12
13	Database Design	12
14	Website Analyzer	13
15	Responsive Design, Accessibility and Security	14
16	Experiment Implementation Details I	14
17	Experiment Implementation Details II	15
18	Testing and Verification	15
19	XAMPP Runtime Environment	16
20	Local Server and Source-Control Workflow	16
21	Database Evidence — Beneficiaries and Bills	17
22	Database Evidence — Transactions and Users	18
23	Advantages, Limitations and Troubleshooting	18
24	Restart and Shutdown Operating Checklist	19
25	References and Repository Artifacts	20

1. INTRODUCTION

DigitalPay is a full-stack digital payment management system created as an academic simulation. It presents the visual and operational flow of a modern payment platform while deliberately avoiding real settlement, banking connectivity and credential collection. The application combines a professional user interface with browser APIs, a PHP backend, a relational MySQL database and repeatable tests.

The project is designed as a single evolving product instead of nine unrelated laboratory exercises. Semantic HTML becomes a responsive dashboard; JavaScript adds state and events; PHP/MySQL introduces persistent CRUD; and jQuery/AJAX completes the dynamic transaction experience.

1.1 Problem Statement

Payment applications contain many connected journeys: account creation, wallet monitoring, transfers, bills, beneficiaries, transaction search and reporting. Demonstrating these journeys safely in a classroom requires realistic interactions without collecting actual financial information.

1.2 Proposed Solution

DigitalPay provides a coherent fictional environment with reusable components, visible validation, clear status feedback and security boundaries. All names, balances, transaction references and contact details are demo data.

> **Core principle:** realistic full-stack engineering, fictional financial data and explicit safety boundaries.

2. ABSTRACT, OBJECTIVES AND SCOPE

2.1 Abstract

The system implements a responsive fintech interface that demonstrates wallet management, simulated payments, bill organization, beneficiaries, JSON-backed transactions, reports, PHP/MySQL user administration and guarded web-page analysis. Shared navigation and visual tokens make the individual experiments feel like one deployable application.

2.2 Objectives

- Develop an accessible, responsive interface with semantic HTML and a maintainable CSS design system.
- Demonstrate event-driven JavaScript, drag/drop, Web Storage, jQuery and AJAX.
- Implement validated PHP endpoints and relational MySQL CRUD using PDO prepared statements.
- Provide a guarded analyzer for public webpages with clear error handling.
- Document installation, testing, Git workflow and day-to-day operation.

2.3 Scope

The scope is limited to academic demonstration. Authentication, real payment processing, bank or UPI integration, settlement, production audit trails and card-data storage are intentionally excluded.

2.4 Measurable Outcomes

METRIC	VALUE
Laboratory experiments represented	9
Primary application screens	8
Relational database tables	4
AJAX JSON records	8
Automated smoke-test checks	34

3. TECHNOLOGY STACK

LAYER	TECHNOLOGY	ROLE IN DIGITALPAY
Structure	HTML5	Semantic landmarks, forms, tables, native validation and accessible controls.
Presentation	CSS3	Variables, Grid, Flexbox, responsive breakpoints, animation and print styling.
Client logic	JavaScript	Counters, theme, events, drag/drop, Web Storage and interactive state.
Dynamic UI	jQuery 3.7.1	Selection, filtering, delegated events and AJAX rendering.
Data exchange	JSON / AJAX	Eight fictional transaction records loaded over HTTP.
Server	PHP 8.1+	Validation, CRUD, JSON API, uploads and website analysis.
Database	MySQL 8 / PDO	Users, transactions, beneficiaries and bills with relational constraints.
Runtime	XAMPP / PHP server	Local Apache, PHP and MySQL development environment.
Quality	Node.js smoke test	Dependency-free verification of files, data and required implementations.
Version control	Git / GitHub	Source history and public project repository.

Design language: a dark navy fintech palette, blue action color, cyan accents, rounded cards, strong focus states and consistent typography across every module.

4. SYSTEM ARCHITECTURE

PRESENTATION	CLIENT LOGIC	SERVER & DATA
HTML screens Shared CSS Responsive layout Accessible controls	JavaScript modules jQuery selectors AJAX requests Web Storage	PHP endpoints PDO prepared SQL MySQL database Public-page analyzer
Home · Dashboard · Registration · Wallet · Payments · Transactions · Reports	DOM events → state update → visible feedback	Validated request → prepared query → escaped response

4.1 Request Flow

The browser renders static modules and performs local interactions immediately. AJAX loads JSON or calls PHP endpoints over HTTP. PHP validates input, uses environment-based database configuration and accesses MySQL through PDO. Responses are escaped for HTML or encoded as JSON before returning to the browser.

4.2 Folder Organization

The project separates concerns into css/, js/, data/, php/, database/, images/, screenshots/ and tests/. Root-level pages compose the user experience, while reusable assets keep behavior and visual styling consistent.

4.3 Data Boundaries

Temporary notes use sessionStorage; wallet queues and theme preference use localStorage; AJAX examples use a versioned JSON file; user and transaction records use MySQL. This separation makes storage lifetimes and trust boundaries explicit.

5. HOME PAGE AND PRODUCT IDENTITY

The landing page introduces the platform, developers, feature set, safety boundary and shared navigation.

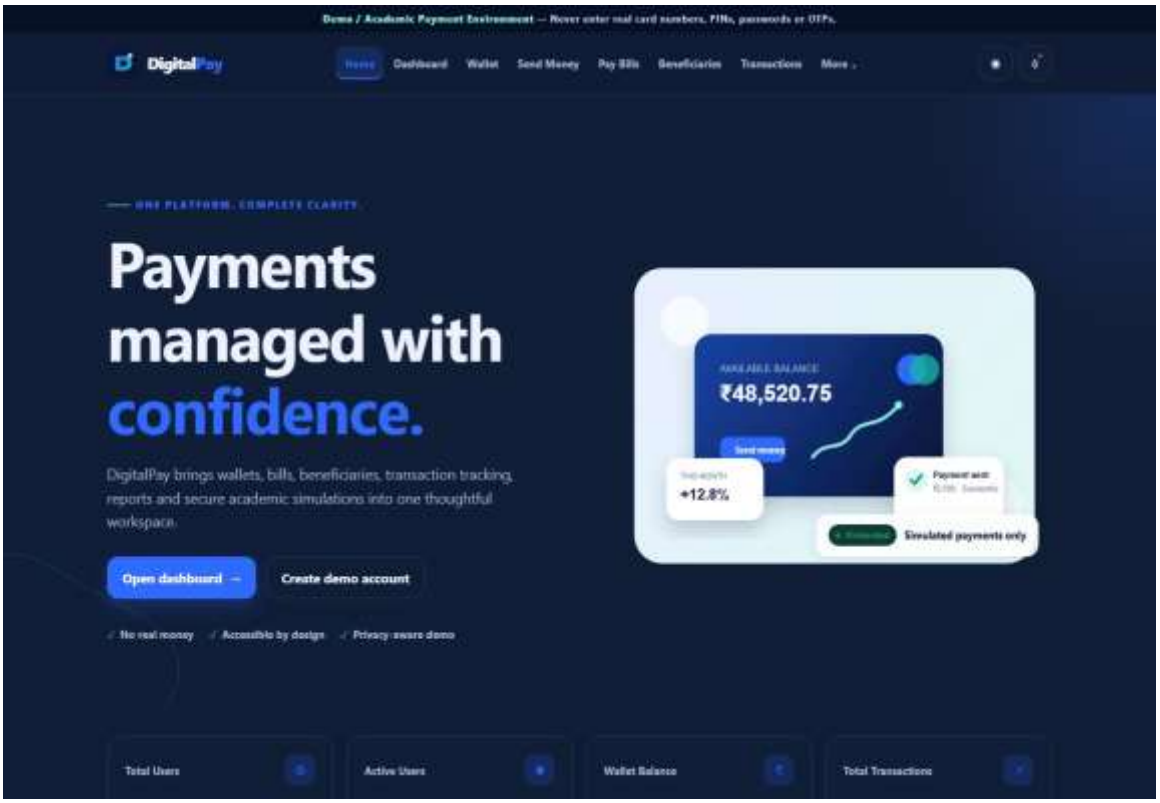


Figure 5.1 — DigitalPay home page captured from http://localhost:8000/

The hero communicates the payment-management theme without implying real banking. Statistics, service cards, project identity, experiment modules, security guidance and academic contact areas continue below the fold.

6. INTERACTIVE DASHBOARD

The dashboard consolidates wallet balance, transaction totals, successful-payment rate, pending work, quick actions and monthly goals.

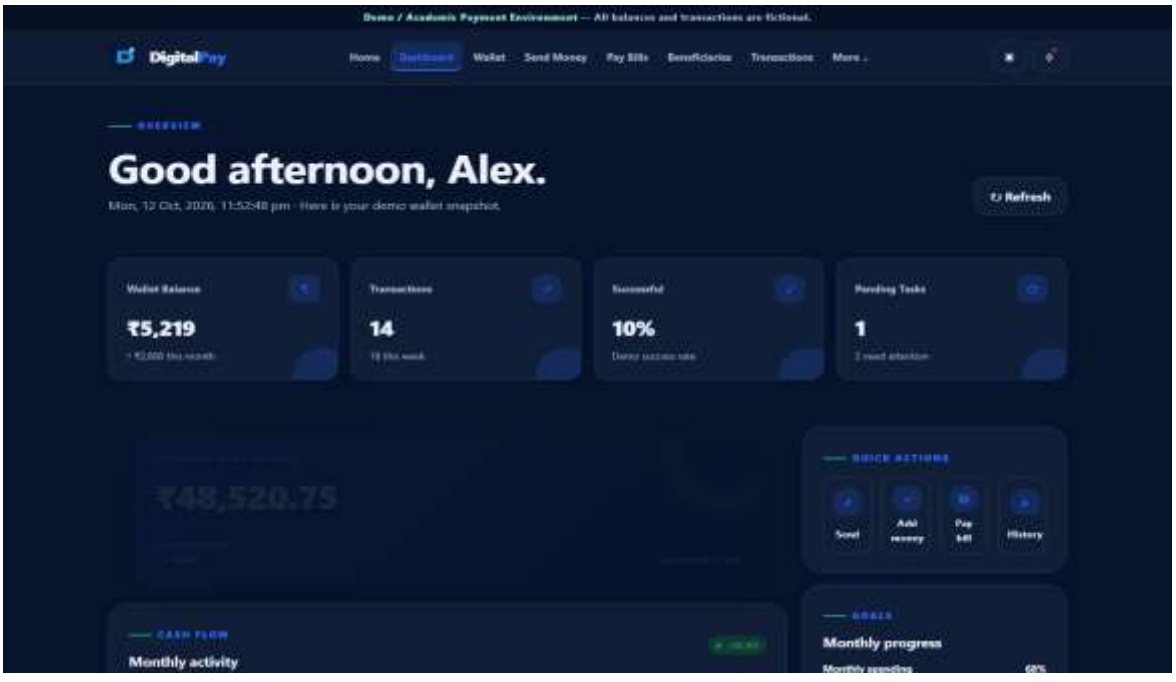


Figure 6.1 — Interactive dashboard with fictional live metrics

JavaScript initializes the live greeting and date, updates visual counters, persists the theme and supports notifications and quick actions. The displayed name and balances are fictional demonstration data.

7. ADVANCED REGISTRATION

The registration module demonstrates native HTML5 controls, validation constraints and an accessible information hierarchy.

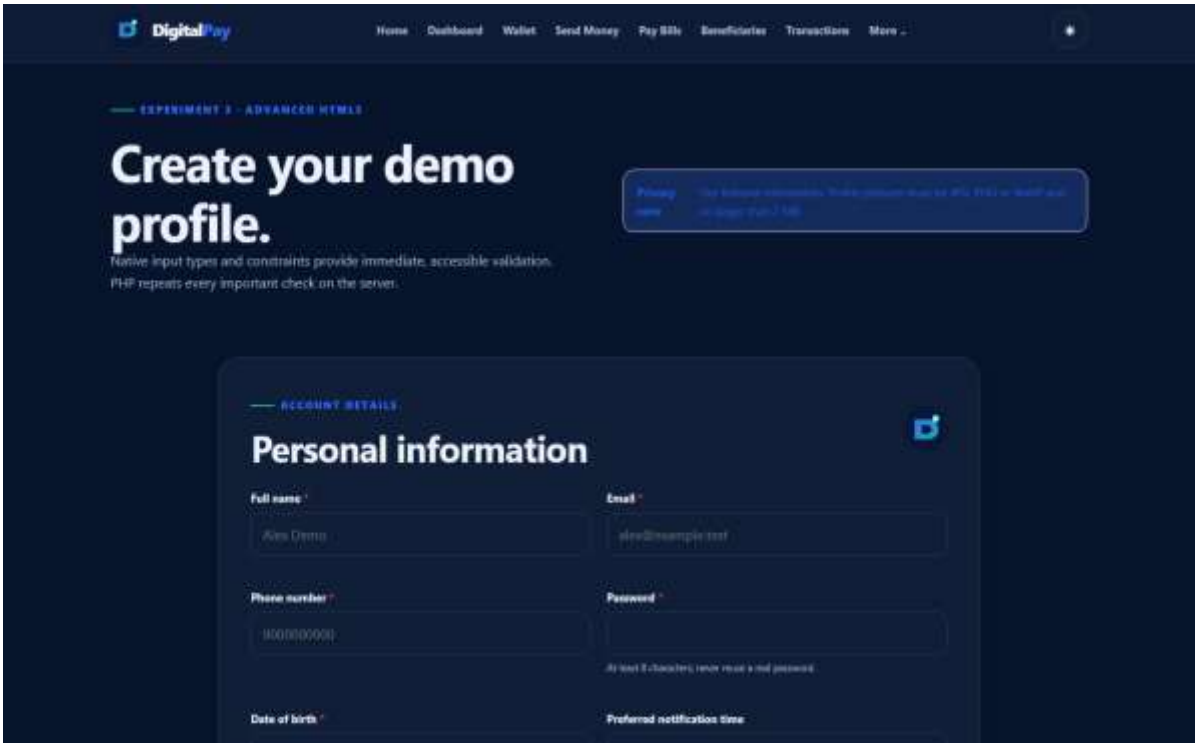


Figure 7.1 — Advanced profile registration screen

The form uses email, telephone, date, time, number and file controls together with required, pattern, min/max and file-type rules. Client feedback improves usability, while PHP repeats validation before any database change.

8. WALLET AND PAYMENT QUEUE

The wallet experiment combines drag/drop interaction with persistent and temporary browser storage.

Services can be dragged into a queue or added by keyboard. Save and Retrieve demonstrate localStorage persistence across reloads; the temporary note demonstrates sessionStorage for the current browser tab.

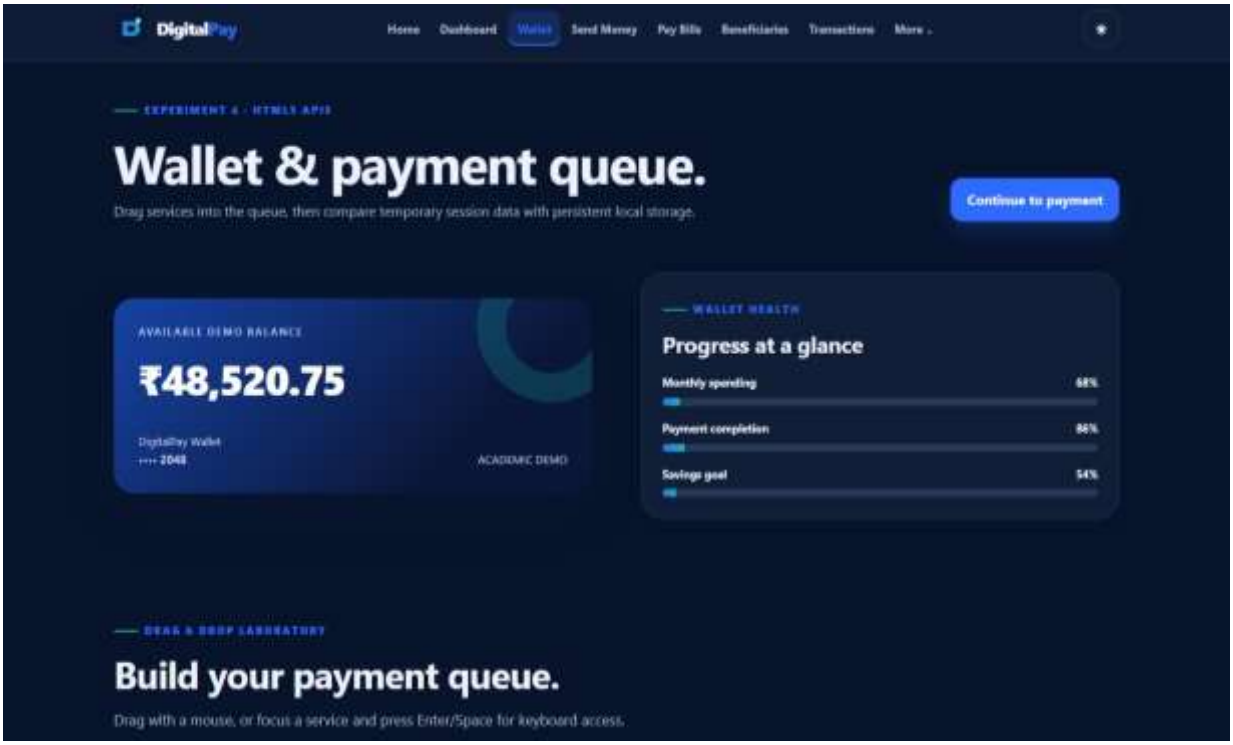


Figure 8.1 — Wallet balance, progress indicators and payment queue

9. EVENT-DRIVEN PAYMENT CENTER

Payments, bills, QR-style simulation and beneficiaries share an event-driven interface.

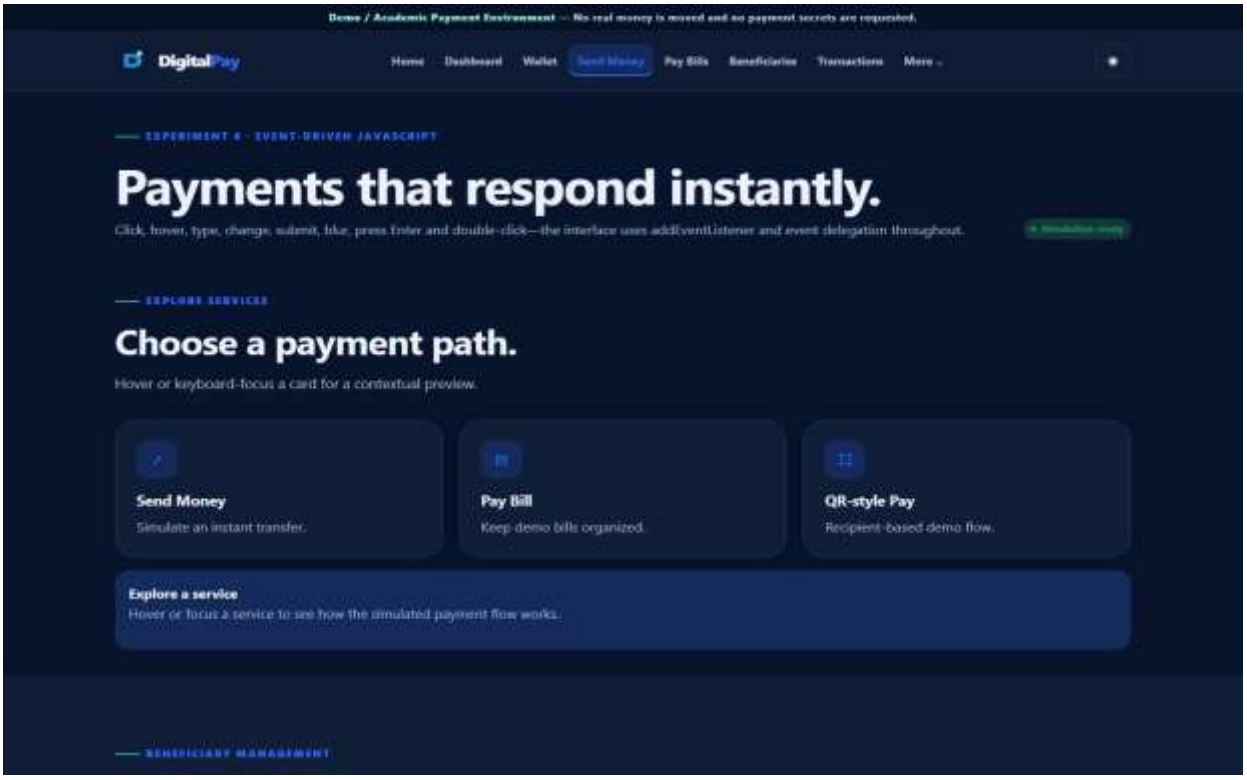


Figure 9.1 — Event-driven payment and beneficiary center

The module handles click, hover, input, change, submit, blur, keydown and double-click events. Delegated controls continue to work for dynamically created beneficiaries, and form submission generates fictional transaction feedback only.

10. TRANSACTION CENTER

The jQuery transaction center demonstrates selection, compound filtering, dynamic records and AJAX loading.

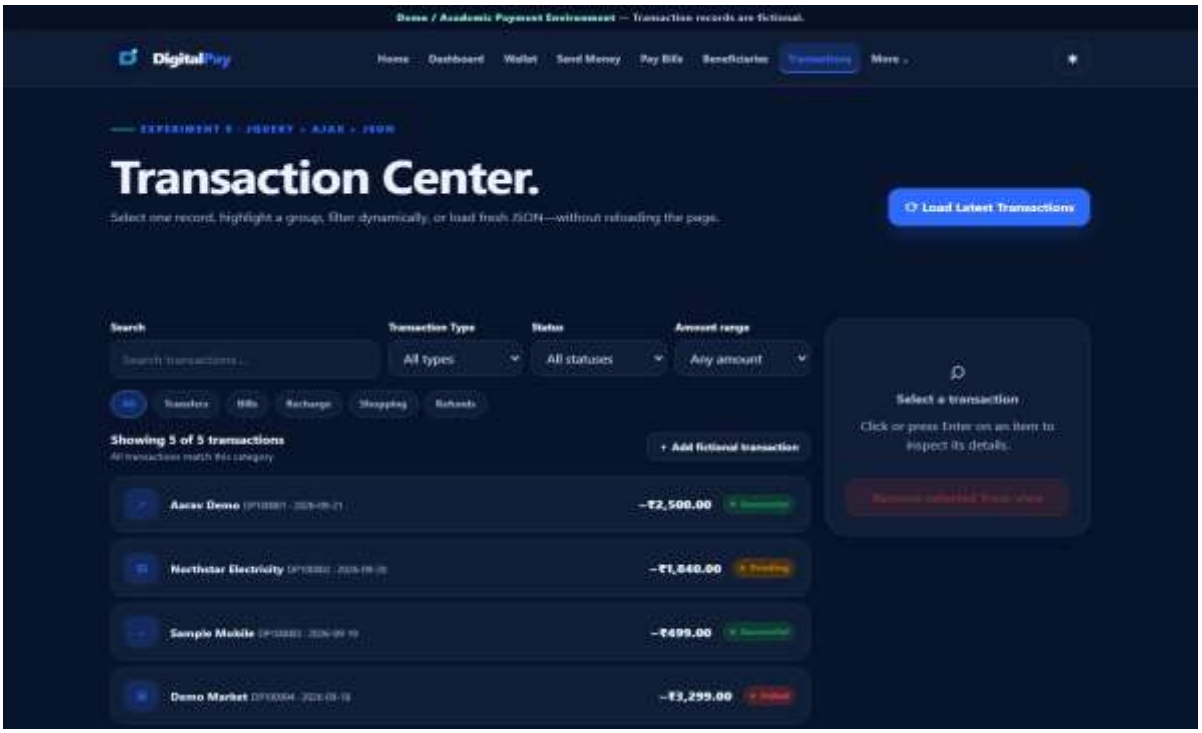


Figure 10.1 — Searchable and filterable transaction center

Every record has a unique identifier, class and data attributes. Users can inspect one record, highlight a category, combine type/status/amount filters, add or remove a fictional item and load eight JSON records with a retry path.

11. REPORTS AND INSIGHTS

The report screen summarizes fictional outflow, movement, bills, success rate and category distribution.



Figure 11.1 — Print-friendly payment report

Responsive charts and KPI cards translate the shared transaction data into readable trends. A print stylesheet removes unnecessary navigation and produces a clean submission-friendly report view.

12. PHP BACKEND AND USER MANAGEMENT

PHP endpoints supply validated persistence and administrative CRUD while preserving the same visual system.

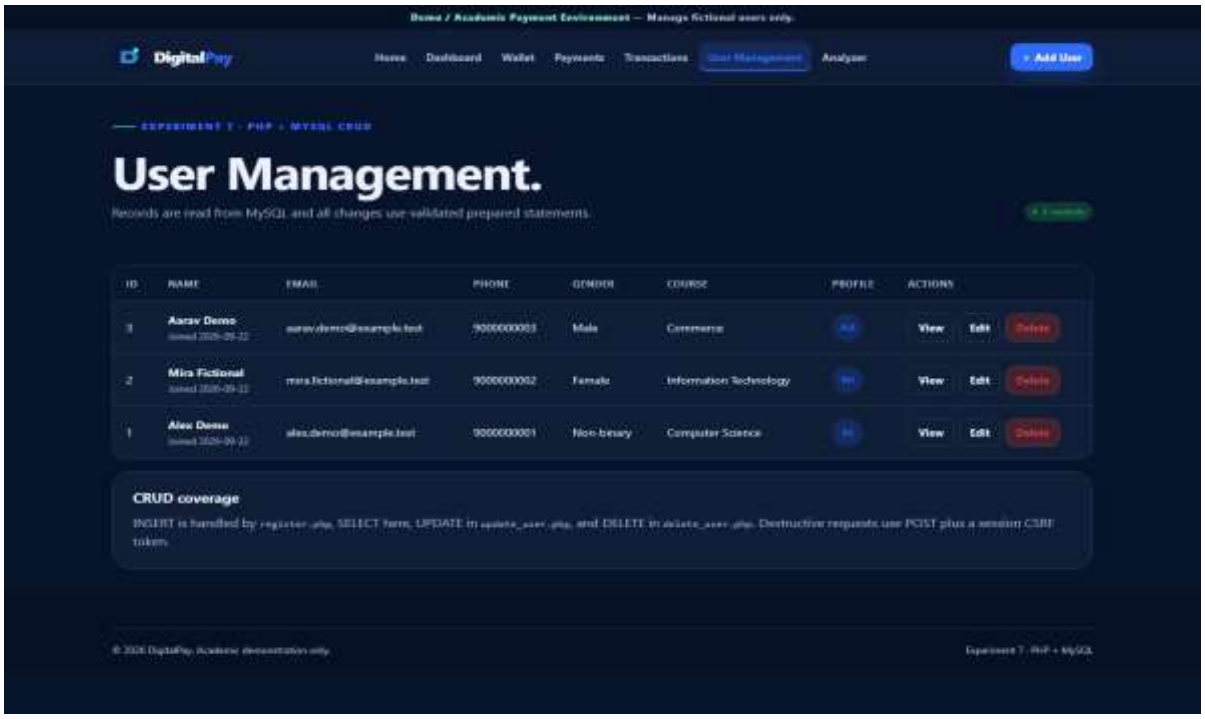


Figure 12.1 — MySQL-backed User Management screen

ENDPOINT	RESPONSIBILITY
php/register.php	Validate fields, prevent duplicates, hash passwords, restrict uploads and INSERT.
php/users.php	SELECT demo profiles and render the administration table.
php/update_user.php	View and UPDATE a selected user through prepared statements.
php/delete_user.php	POST-only, CSRF-protected prepared DELETE.
php/transactions.php	Return MySQL transaction data as JSON.
php/config.php	Environment-based PDO connection, escaping and CSRF helpers.

13. DATABASE DESIGN

TABLE	PRIMARY PURPOSE	IMPORTANT CONSTRAINTS
users	Profile and academic information	Unique email/phone; ten-digit phone check; password hash.
transactions	Payment activity and references	FK to users; positive amount; unique reference; status enum; indexes.
beneficiaries	Saved fictional recipients	FK to users; unique account identifier per user.
bills	Utility and subscription queue	FK to users; positive amount; due-date and status indexes.

13.1 Relationship Model

One user can own many transactions, beneficiaries and bills. Foreign keys use cascading delete and update behavior so dependent demo records remain consistent. utf8mb4 supports full Unicode text.

13.2 Data Integrity

Unique keys prevent duplicate identity fields and transaction references. CHECK constraints reject invalid phone or amount values. Indexes support common user/date and status lookups.

13.3 Fictional Seed Data

The schema supplies three demo users, eight transactions, four beneficiaries and five bills. All names, addresses, references and balances are fictional and use example.test-style addresses.

Configuration: database credentials are read from DIGITALPAY_DB_* environment variables. Credentials and uploaded profile images are excluded from version control.

14. WEBSITE ANALYZER

The analyzer inspects public HTML metadata and resources while applying strict network and content controls.

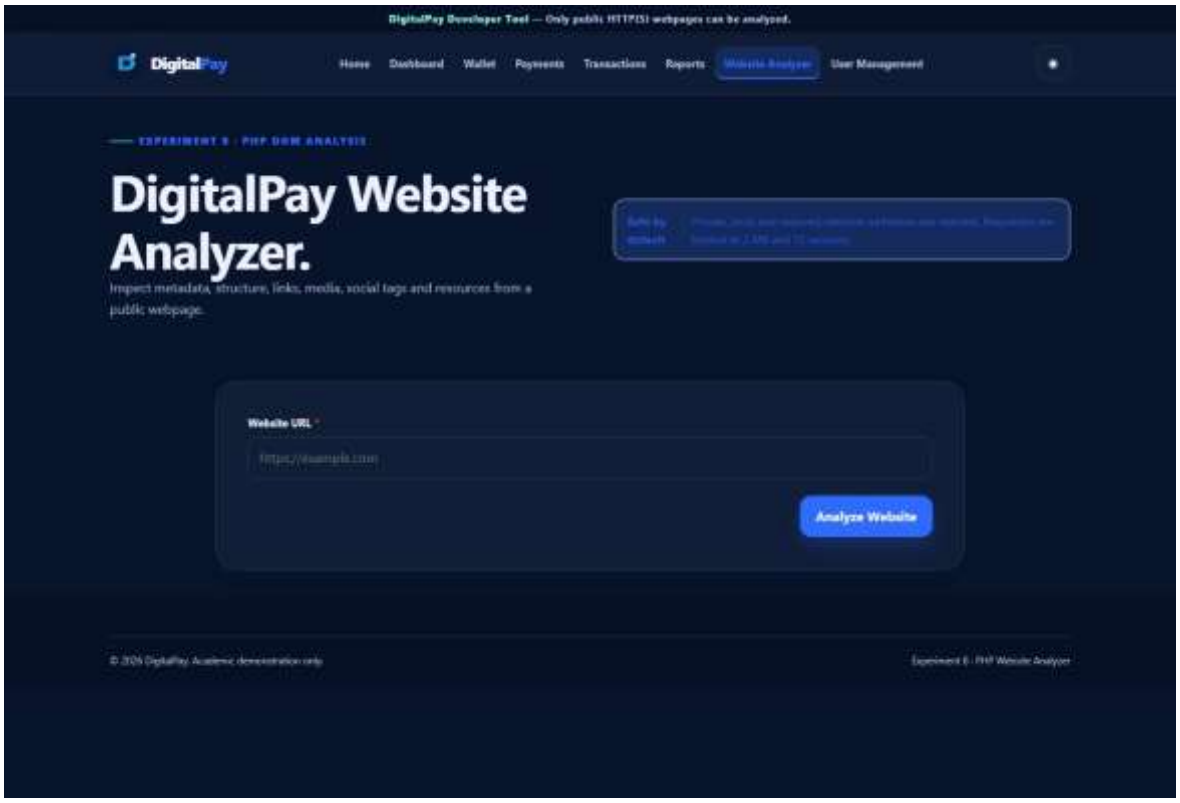


Figure 14.1 — DigitalPay guarded website analyzer

The PHP engine validates HTTP(S) URLs, resolves every destination and redirect, blocks private or reserved network targets, verifies TLS, enforces time and body-size limits and accepts HTML content only. DOMDocument and XPath extract title, description, keywords, headings, links, images, stylesheets, scripts and forms.

Security boundary: the analyzer is a learning tool for small public pages. It does not execute fetched scripts and must not be used to access private-network resources.

15. RESPONSIVE DESIGN, ACCESSIBILITY AND SECURITY

15.1 Responsive Design

CSS variables, Grid and Flexbox create consistent layouts at desktop, tablet and mobile sizes. Breakpoints reorganize navigation, cards, forms, tables and transaction panels without horizontal page overflow. Print rules support clean report output.

15.2 Accessibility

Semantic landmarks, visible labels, skip navigation, keyboard-operable controls, focus indicators, meaningful alternative text and reduced-motion support improve access. Drag/drop actions have keyboard alternatives and messages provide visible status feedback.

15.3 Security Controls

- PDO emulated prepares are disabled and every mutation uses bound prepared statements.
- Passwords are hashed; duplicate email and phone values are checked server-side.
- CSRF tokens protect user update/delete operations and mutations use POST.
- Image uploads are restricted by type and size, renamed randomly and ignored by Git.
- The website analyzer blocks SSRF-style private targets and constrains redirects, time and size.
- Output is escaped and the project explicitly rejects real payment secrets.

> Production would additionally require authentication, role-based authorization, rate limits, audit logs, CSP, malware scanning, migrations, monitoring and automated integration tests.

16. EXPERIMENT IMPLEMENTATION DETAILS I

EXP.	IMPLEMENTATION	DEMONSTRATION
1	index.html	Semantic header/nav/main/sections/footer, eight statistics, lists, form, module table and contact.
2	style.css, main.js, dashboard.js	Grid/Flexbox, animations, live date/time, counters, theme, notifications, slider and validation.
3	register.html	Specialized HTML5 controls, constraints, contenteditable, spellcheck and accessible validation.
4	wallet.html, storage.js	dragstart/dragover/drop, keyboard add, localStorage, sessionStorage and queue controls.
5	Shared CSS and all screens	Breakpoints, pseudo-elements, transforms, transitions, progress bars, tooltips and print output.

Demonstration Sequence

- Open Home and identify semantic regions, statistics, lists, forms and tables.
- Toggle the theme, notifications and responsive navigation; observe persisted preference.
- Submit invalid registration fields to demonstrate browser constraints.
- Add services to the wallet queue, save, reload and retrieve.
- Resize the viewport to desktop, tablet and mobile widths and inspect layout changes.

17. EXPERIMENT IMPLEMENTATION DETAILS II

EXP.	IMPLEMENTATION	DEMONSTRATION
6	payments.html, events.js	Eight event types, preventDefault, delegation, beneficiary filtering and fictional payment feedback.
7	database/, php/	Schema import, validated registration and prepared INSERT/SELECT/UPDATE/DELETE.
8	analyzer.php, php/analyzer.php	Public URL inspection, metadata extraction, URL resolution and guarded error handling.
9	transactions.html, jquery-app.js, JSON	Selection, group highlighting, filters, dynamic add/remove and $.ajax loading with Retry.

Demonstration Sequence

- Search and favorite a beneficiary, add a dynamic recipient and submit a demo transfer.
- Register a fictional user, then view, edit and delete it from User Management.
- Analyze a small public page, followed by malformed and unreachable URL cases.
- Select transaction cards, combine filters, add/remove a record and reload JSON through AJAX.

Outcome

Together, the nine experiments demonstrate progressive enhancement from document structure to a coherent client/server/database system with repeatable verification.

18. TESTING AND VERIFICATION

TEST AREA	CHECKS PERFORMED	EXPECTED RESULT
Required files	25 key pages/assets/endpoints	Every required file exists.
JSON	Array shape and six required fields	Eight complete records parse successfully.
Events	addEventListener and no inline onclick	Event-driven implementation confirmed.
Storage	localStorage, sessionStorage and drop	Both lifetimes and drag/drop are present.
jQuery	$.ajax and delegated selection	Dynamic AJAX behavior confirmed.
SQL	Users table and foreign keys	Schema and relationships confirmed.
PHP	prepare() in CRUD/API files	Prepared statements confirmed.

Automated result: DigitalPay smoke tests passed (34 checks).

18.1 Manual Verification

- Form errors: empty, malformed email, short phone, invalid amounts and short passwords.
- Browser behavior: theme, clock, counters, notifications, slider, queue and storage lifetime.
- Backend: database import, duplicate rejection, update/delete cancellation and database-offline message.
- Analyzer: valid page, malformed URL, blocked private address, non-HTML, timeout and large response.
- AJAX: successful load, missing/invalid JSON error state and Retry behavior.

19. XAMPP RUNTIME ENVIRONMENT

XAMPP provides the local Apache, PHP and MySQL services used for full-stack development and demonstration.

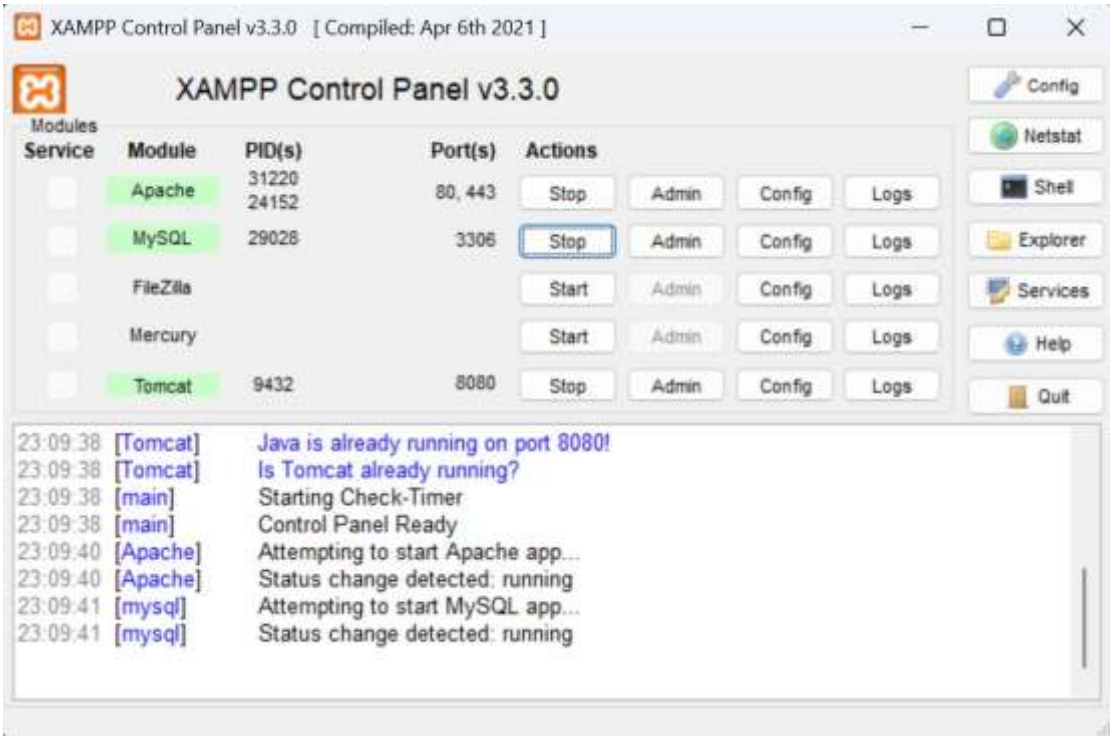


Figure 19.1 — Supplied XAMPP Control Panel evidence showing Apache and MySQL running

The evidence shows Apache on ports 80/443 and MySQL on port 3306. Tomcat is also running on port 8080 but is not required by DigitalPay. For this project, Apache/PHP serves the pages and endpoints while MySQL stores persistent user, transaction, beneficiary and bill data.

20. LOCAL SERVER AND SOURCE-CONTROL WORKFLOW

The application is served locally by PHP and uses Git/GitHub for source history and submission.

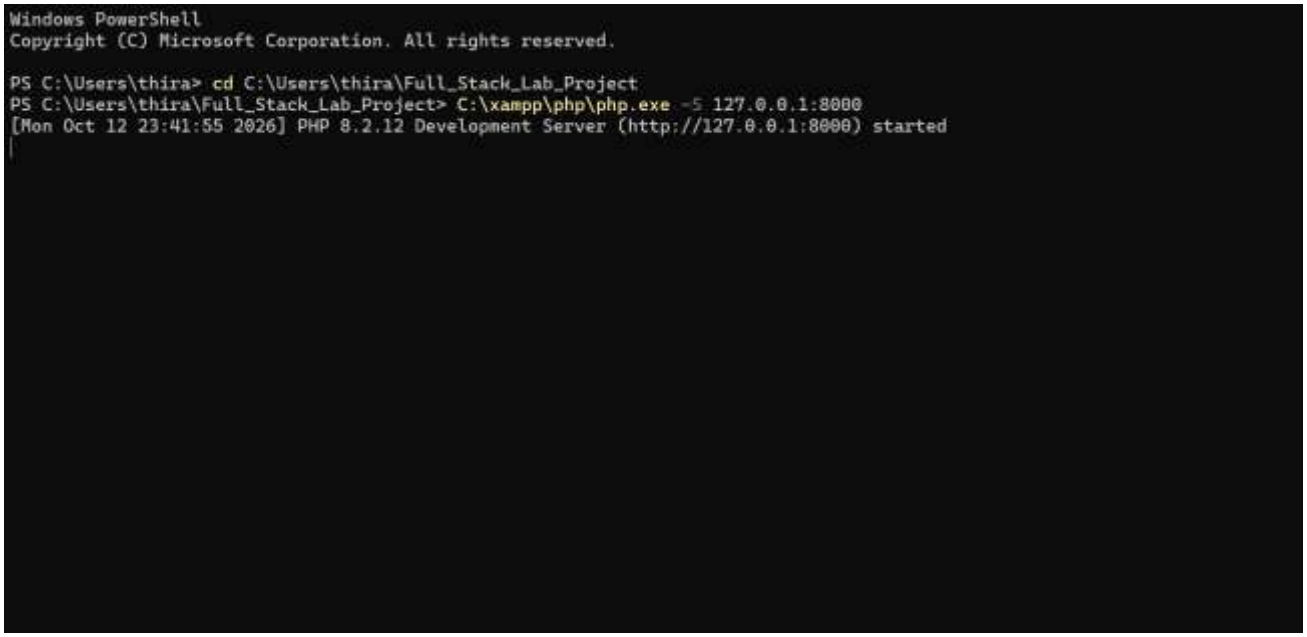


Figure 20.1 — Supplied PowerShell evidence: PHP 8.2.12 development server on 127.0.0.1:8000

The command window confirms that the working directory is the DigitalPay project and that PHP serves the application at http://127.0.0.1:8000. Git records local changes, while the public repository preserves the submitted source structure: CSS, JavaScript, PHP, database, JSON data, images, screenshots and tests.

Repository: https://github.com/ThirajVS/digitalpay

21. DATABASE EVIDENCE — BENEFICIARIES AND BILLS

The supplied phpMyAdmin screens confirm that the implemented relational schema contains persisted demonstration records.

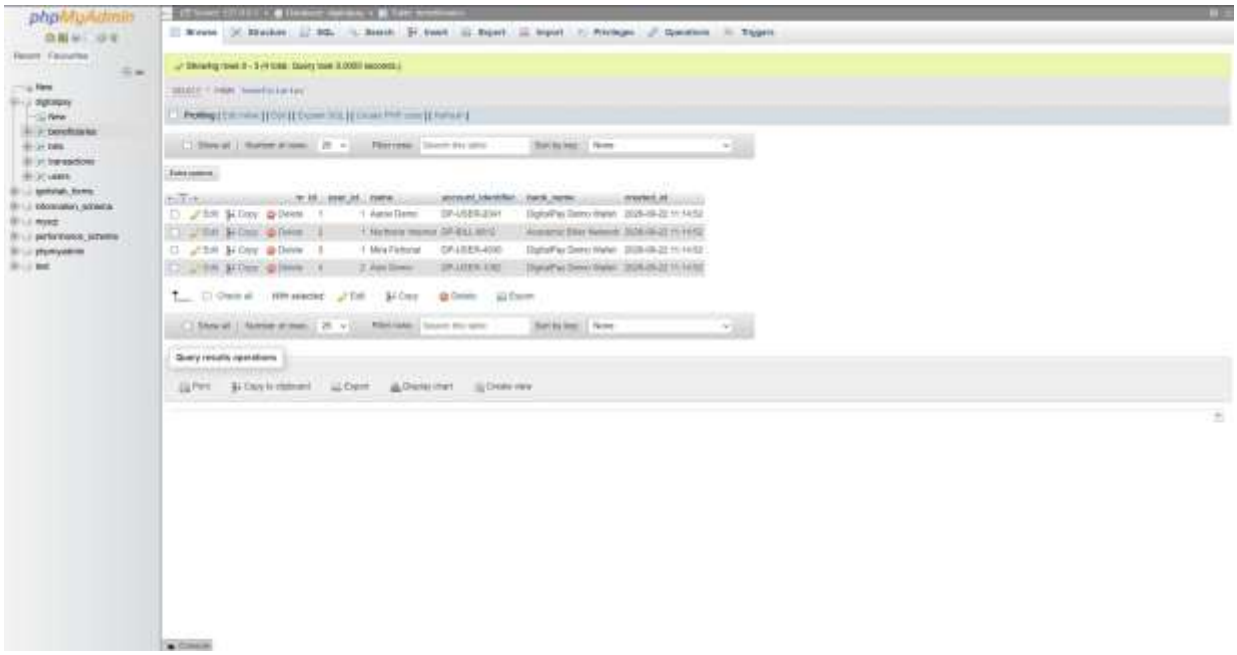


Figure 21.1 — Beneficiaries table with user relationships and fictional account identifiers

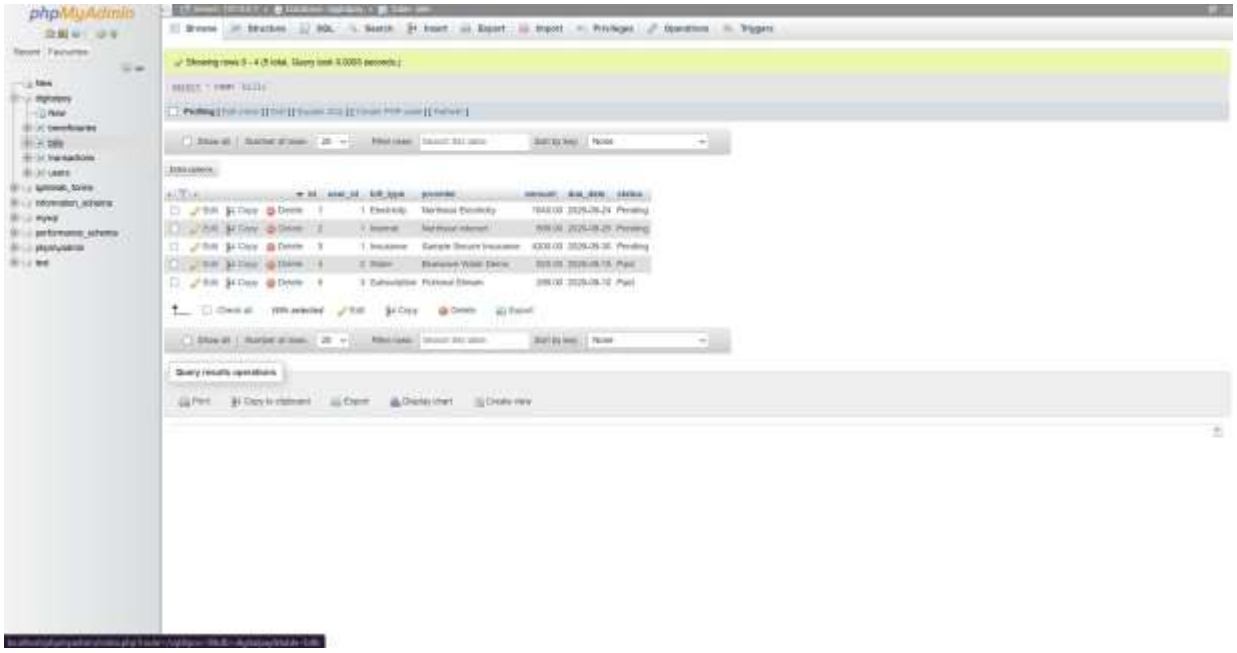


Figure 21.2 — Bills table with providers, positive amounts, due dates and status values

22. DATABASE EVIDENCE — TRANSACTIONS AND USERS

The database stores fictional transaction activity and user profiles; password values are hashes rather than plain text.

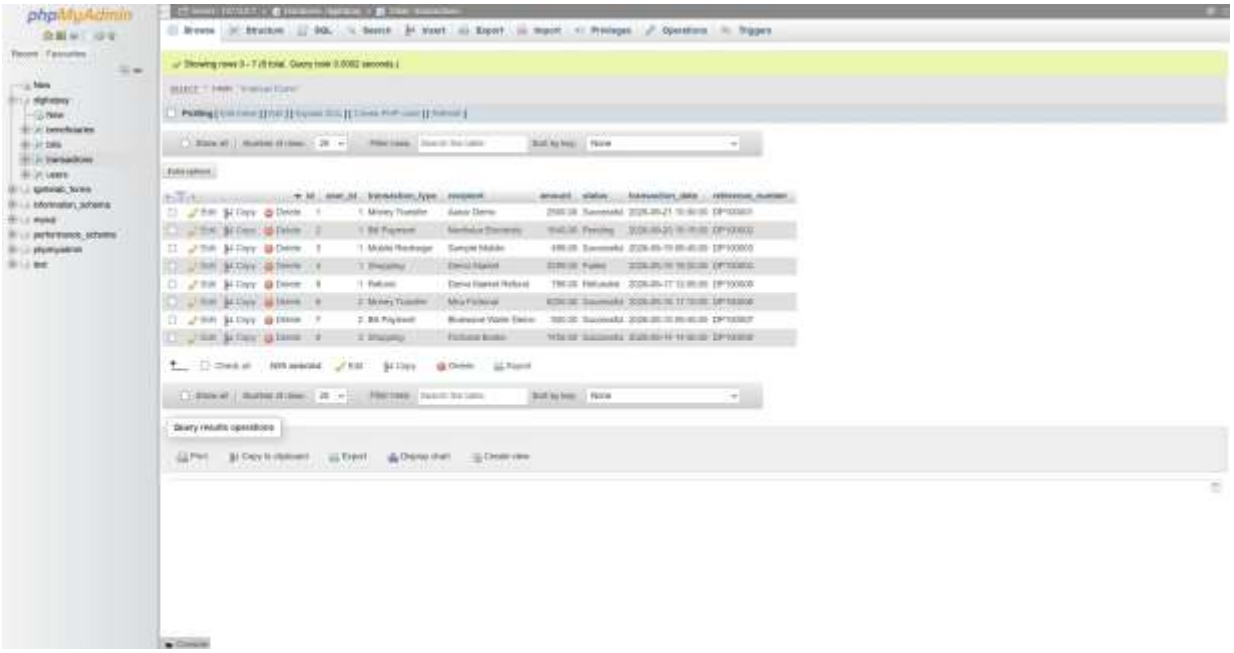


Figure 22.1 — Transactions table with type, recipient, amount, status, date and reference number

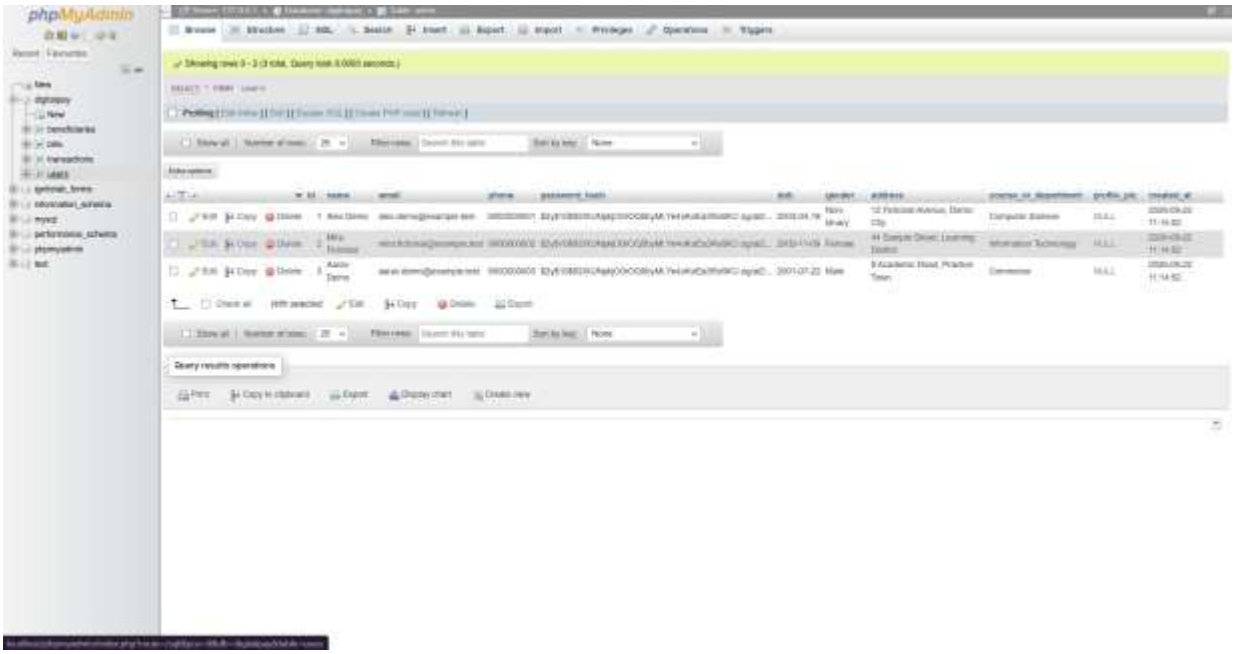


Figure 22.2 — Users table with unique identity fields and hashed passwords

23. ADVANTAGES, LIMITATIONS AND TROUBLESHOOTING

ADVANTAGES	CURRENT LIMITATIONS
One coherent product for nine experiments	No real payment processing
Responsive, keyboard-aware interface	No production authentication
Prepared CRUD and guarded analyzer	Local PHP/MySQL services required
Repeatable 34-check smoke test	No bank or UPI integration

23.1 Troubleshooting Guide

SYMPTOM	ACTION
localhost does not open	Start Apache/PHP and confirm the port.
User Management fails	Start MySQL and import digitalpay.sql.
AJAX records do not load	Use HTTP rather than file://.
Analyzer rejects a URL	Use a public HTTP(S) HTML page.

23.2 Conclusion

DigitalPay successfully combines semantic structure, responsive design, browser interaction, persistent storage, jQuery/AJAX, PHP/MySQL CRUD and guarded server-side analysis in one safe academic payment-management system.

23.3 Future Enhancements

- Add authenticated sessions, roles, audit logs and rate limiting.
- Introduce migrations, CI integration and browser tests.
- Provide Docker deployment, HTTPS and structured monitoring.
- Add localization and accessible CSV/PDF exports.

24. RESTART AND SHUTDOWN OPERATING CHECKLIST

STEP	ACTION	DETAIL
1	Start XAMPP	Start Apache and MySQL; wait for green status labels.
2	Open PowerShell	Use C:\Users\thira\Full_Stack_Lab_Project.
3	Start port 8000	Run php -S 127.0.0.1:8000 when using the PHP development server.
4	Open DigitalPay	Browse to http://localhost:8000/.
5	Verify backend	Open User Management and confirm MySQL records load.
6	Run checks	Execute npm test and confirm 34 checks pass.
7	Stop safely	Use Ctrl+C for the PHP server; stop XAMPP services when no longer needed.

If the page does not load, first confirm the correct web server is running, then check the selected port and the PowerShell output. Do not delete the database, project folder or MySQL data during normal recovery.

25. REFERENCES AND REPOSITORY ARTIFACTS

NO.	REFERENCE
1	MDN Web Docs — HTML forms, CSS layout, JavaScript events, Drag and Drop, and Web Storage.
2	PHP Manual — PDO, password hashing, cURL, DOMDocument and secure upload handling.
3	MySQL 8.0 Reference Manual — constraints, indexes, foreign keys and transactions.
4	jQuery API Documentation — selectors, delegated events and $.ajax.
5	OWASP guidance — input validation, CSRF prevention, output encoding, SSRF prevention and file uploads.
6	Git and GitHub documentation — commits, remotes, branches and repository hygiene.
7	DigitalPay project source and README — https://github.com/ThirajVS/digitalpay

ARTIFACT	LOCATION / DESCRIPTION
Project repository	https://github.com/ThirajVS/digitalpay
Primary documentation	README.md
Frontend source	Root HTML pages, css/ and js/
Backend source	analyzer.php and php/ endpoints
Database	database/digitalpay.sql
JSON data	data/transactions.json
Automated verification	tests/smoke-test.mjs

This report was prepared from the implemented DigitalPay repository, the running local application and the supplied XAMPP and GitHub evidence. Completed work is separated from proposed production enhancements.